

#05 Number Theory I & Base Conversion

김성수

4thrunyul@gmail.com

0. 목차



1. Number Theory I

- Modular Arithmetic
- Finding Divisors
- GCD & LCM
- Euclidean Algorithm
- Primality Test
- Sieve of Eratosthenes

2. Base Conversion

- Decimal to Base-N
- Base-N to Decimal
- Horner's Method

1. Number Theory I

1.1 Modular Arithmetic

Modular Arithmetic(나머지 연산, mod 연산)

- 정수 a , n 이 있을 때, $a \bmod n$ 은 a 를 n 으로 나눴을 때의 나머지
- $7 \bmod 3 = 1$
- $10 \bmod 5 = 0$
- $3 \bmod 10 = 3$
- $a \bmod n$ 의 결과가 r 이라면 0 이상 $n - 1$ 이하의 정수가 되기에 값의 범위를 제한하는 특성이 있음.
- 대표적인 사용 예로 홀수, 짝수 판별이 있음.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9
10     if(n % 2 == 0) cout << "even";
11     else cout << "odd";
12
13     return 0;
14 }
```

입력: 복사
5

예상 출력: 복사
odd

입력: 복사
16

예상 출력: 복사
even

1.1 Modular Arithmetic



나머지 연산의 분배 법칙

- 나머지 연산은 덧셈, 뺄셈, 곱셈의 분배 법칙이 성립함.
- 덧셈
 - $(A + B) \% C = (A \% C + B \% C) \% C$
 - $(20 + 6) \% 3 = 26 \% 3 = 2$
 - $(20 \% 3 + 6 \% 3) \% 3 = (2 + 0) \% 3 = 2$
- 뺄셈
 - $(A - B) \% C = (A \% C - B \% C) \% C$
 - $(20 - 6) \% 3 = 14 \% 3 = 2$
 - $(20 \% 3 - 6 \% 3) \% 3 = (2 - 0) \% 3 = 2$
- 곱셈
 - $(A * B) \% C = (A \% C * B \% C) \% C$
 - $(20 * 6) \% 3 = 120 \% 3 = 0$
 - $(20 \% 3 * 6 \% 3) \% 3 = (2 * 0) \% 3 = 0$

1.1 Modular Arithmetic



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      long long sum = 1;
7
8      for(int i = 1; i <= 50; i++) {
9          sum *= i;
10     }
11
12     cout << sum % 1234567891;
13
14     return 0;
15 }
```

Input: Copy

Expected Output: Copy

-1076673018

50!를 1234567891로 나눈 나머지를 구하려고 한다.

해당 코드가 오류가 발생하는 이유는?

- 21!에서 이미 long long 범위를 벗어나서 overflow 문제가 발생함

오류를 바로잡고 원하는 결과를 얻기 위해 수정해야 할 로직은?

1.1 Modular Arithmetic



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      long long sum = 1;
7
8      for(int i = 1; i <= 50; i++) {
9          sum = (sum * i) % 1234567891;
10     }
11
12     cout << sum;
13
14     return 0;
15 }
```

Input: Copy

Expected Output: Copy
1068536508

50!를 1234567891로 나눈 나머지를 구하려고 한다.

수정된 로직

- 연산이 완료된 시점에서 나머지 연산을 수행하는 것이 아니라 연산이 수행 되는 도중에 계속 나머지 연산을 수행함.
- 나머지 연산의 분배 법칙에 근거해 원하는 결과값을 얻을 수 있음.

1.1 Modular Arithmetic



실습(BOJ)

- [33046 Alea lacta Est](#)
- [15829 Hashing](#)

1.1 Modular Arithmetic



실습(정올)

- [2856 10부제](#)
- [19539 산술 문제](#)
- [2911 사과](#)
- [1009 각 자리수의 역과 합](#)

1.2 Finding Divisors



약수(Divisor)

- 두 정수 a 와 b 에 대해 $a = bk$ 를 만족하는 정수 k 가 존재할 때, ' b 는 a 의 약수다'라고 정의함.

1.2 Finding Divisors

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9
10     for(int i = 1; i <= n; i++) {
11         if(n % i == 0) cout << i << " ";
12     }
13
14     return 0;
15 }
```

Input: Copy

10

Expected Output: Copy

1 2 5 10

약수 판별

- 어떤 정수 N이 있을 때 정수 i로 나누어 떨어진다면 약수
- 해당 코드에서는 1 ~ N까지 순회를 통해 나머지가 0이라면 출력

1.2 Finding Divisors

소인수분해

- 1보다 큰 자연수를 소수(Prime Number)들의 곱으로만 나타낸 것
- $2 \sim \sqrt{n}$ 까지 나머지 연산을 실행하여 나머지가 0이라면 출력
- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9
10     for(int i = 2; i * i <= n; i++) {
11         while(n % i == 0) {
12             cout << i << " ";
13
14             n /= i;
15         }
16     }
17
18     if(n > 1) cout << n;
19
20     return 0;
21 }

```

Input:

60

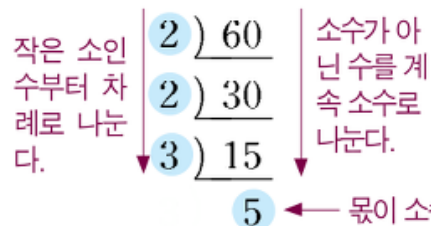
Copy

Expected Output:

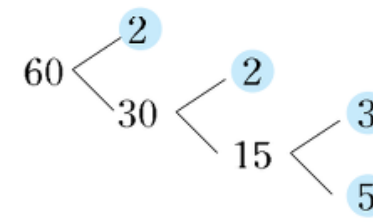
2 2 3 5

Copy

[방법 1]



[방법 2]



↓ 나눈 소수들과 마지막 몫을 곱으로 나타낸다.

소인수분해한 결과

$$60 = 2 \times 2 \times 3 \times 5 = 2^2 \times 3 \times 5$$

60의 소인수 : 2, 3, 5

1.2 Finding Divisors



실습(정올)

- [2809 약수](#)
- [2227 약수의 합](#)
- [2811 소수와 합성수](#)

1.3 GCD & LCM



최대공약수(GCD, Greatest Common Divisor)

- 둘 이상의 정수들의 공통된 약수 중 가장 큰 양수.

최소공배수(LCM, Least Common Multiple)

- 둘 이상의 정수들의 공통된 배수 중에서 가장 작은 양수

GCD와 LCM의 관계

- 정수 A, B가 주어졌을 때,
- $A * B = \text{GCD}(A, B) * \text{LCM}(A * B)$
- $\text{LCM}(A, B) = A * B / \text{GCD}(A, B)$

1.3 GCD & LCM

```

1  #include <iostream>
2  #include <numeric>
3
4  using namespace std;
5
6  int main() {
7      int a, b, result_gcd, result_lcm;
8
9      cin >> a >> b;
10
11     result_gcd = gcd(a, b);
12     result_lcm = lcm(a, b);
13     //result_lcm = a * b / gcd(a, b);
14     //ex) 12 * 18 / 6 = 36;
15
16     cout << result_gcd << " " << result_lcm;
17
18     return 0;
19 }

```

Input:

12 18

Copy

Expected Output:

6 36

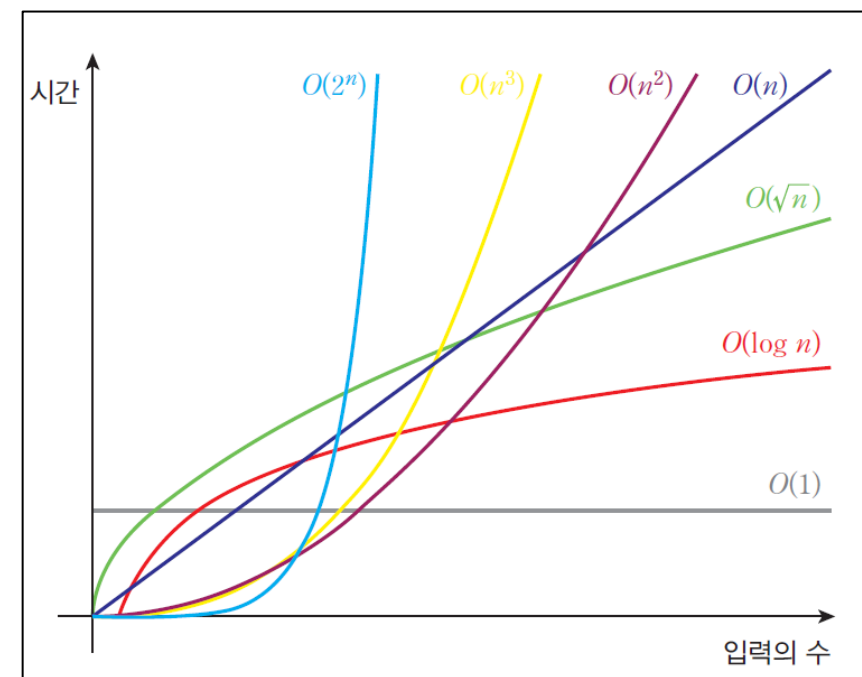
Copy

$\text{gcd}(a, b) \mid \langle \text{numeric} \rangle \mid O(\log n)$

- 두 정수의 최대공약수를 구하는 함수

$\text{lcm}(a, b) \mid \langle \text{numeric} \rangle \mid O(\log n)$

- 두 정수의 최소공배수를 구하는 함수



1.4 Euclidean Algorithm



$\text{GCD}(32, 18)$ 은 쉽게 암산으로 풀이가 가능하다.

그렇다면 $\text{GCD}(12345, 123)$ 을 암산으로 구할 수 있을까?
암산이 아니더라도 어디서부터 계산해야 할지 감이 잡히지 않는다.

더 나아가 큰 수들의 GCD 를 효율적으로 계산하는 방법이 있을까?

→ 유클리드 호제법

1.4 Euclidean Algorithm



Euclidean Algorithm(유클리드 호제법)

- 두 정수 A, B 가 있을 때 (단, $A > B$)
- $A = qB + r$ 이라는 식을 세울 수 있다. ($A \bmod B = r$ 과 같음)
- 이때 $\gcd(A, B) = \gcd(B, r)$ 이다.
- $B = 0$ 이 될때까지 반복하면 된다.

예제 1 : GCD(24, 10)

- $\text{GCD}(24, 10) \rightarrow 24 \bmod 10 = 4$
- $\text{GCD}(10, 4) \rightarrow 10 \bmod 4 = 2$
- $\text{GCD}(4, 2) \rightarrow 4 \bmod 2 = 0$
- $\text{GCD}(2, 0) \rightarrow B$ 가 0이 되었으므로 종료, A 에 남아있는 2가 최대공약수

1.4 Euclidean Algorithm



예제 2 : $\text{GCD}(12345, 123)$

- $\text{GCD}(12345, 123) \rightarrow 12345 \bmod 123 = 45$
- $\text{GCD}(123, 45) \rightarrow 123 \bmod 45 = 33$
- $\text{GCD}(45, 33) \rightarrow 45 \bmod 33 = 12$
- $\text{GCD}(33, 12) \rightarrow 33 \bmod 12 = 9$
- $\text{GCD}(12, 9) \rightarrow 12 \bmod 9 = 3$
- $\text{GCD}(9, 3) \rightarrow 9 \bmod 3 = 0$
- $\text{GCD}(3, 0) \rightarrow B$ 가 0이 되었으므로 종료, A에 남아있는 3이 최대공약수

예제 3 : $\text{GCD}(20260716, 2026)$

1.4 Euclidean Algorithm

유클리드 호제법 | $O(\log n)$

- 재귀 함수를 사용해 두 정수의 최대공약수를 구하기

```
1  #include <iostream>
2
3  using namespace std;
4
5  long long gcd(long long a, long long b) {
6      if(b == 0) return a;
7
8      return gcd(b, a % b);
9  }
10
11 int main() {
12     long long a, b, result;
13
14     cin >> a >> b;
15
16     result = gcd(a, b);
17
18     cout << result;
19
20     return 0;
21 }
```

Input:

20260226 2026

Copy

Expected Output:

2

Copy

1.4 Euclidean Algorithm



실습(BOJ)

- [5086 배수와 약수](#)
- [13241 최소공배수](#)

1.4 Euclidean Algorithm



실습(정올)

- [1071 약수와 배수](#)
- [1658 최대공약수와 최소공배수](#)

1.5 Primality Test

소수(Prime Number)

- 1보다 큰 자연수 중 1과 자기 자신만을 약수로 가지는 수 (ex. 2, 3, 5, 7, 11, 13 ...)
- 즉, 약수의 개수가 2개인 수

소수 판별

- $2 \sim \sqrt{n}$ 까지 나머지 연산을 진행하여 순회가 끝날 때까지 나머지가 0인 케이스가 없다면 소수
- ex) 17의 경우 1~4까지 나머지 연산을 진행함.

1.5 Primality Test

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      bool check = true;
8
9      cin >> n;
10
11     if(n <= 1) check = false;
12
13     for(int i = 2; i * i <= n; i++) {
14         if(n % i == 0) {
15             check = false;
16
17             break;
18         }
19     }
20
21     if(check) cout << n << " is Prime Number";
22     else cout << n << " is not Prime Number";
23
24     return 0;
25 }
```

Input: [Copy](#)

Expected Output: [Copy](#)

2 ~ $\sqrt{n}$ 까지 순회를 통해 소수 판별 | $O(\sqrt{n})$

- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

1.5 Primality Test



실습(BOJ)

- [1978 소수 찾기](#)
- [4134 다음 소수](#)

1.5 Primality Test



실습(정올)

- [1740 소수](#)
- [2813 소수의 개수](#)

1.6 Sieve of Eratosthenes

for문을 통한 소수 판별의 한계

- 시간 복잡도가 $O(\sqrt{n})$ 이므로, 다수의 입력값을 판별해야 하는 경우 $O(n \sqrt{n})$ 으로 변함

Sieve of Eratosthenes(에라토스테네스의 체)

- 정의 : 자연수 n 이하의 모든 소수를 빠르게 찾아내는 방법
- 1. 초기화 : 2부터 n 까지의 모든 숫자를 나열
- 2. 선택 : 아직 지워지지 않은 숫자 중 가장 작은 수(p)를 찾기(p 는 소수)
- 3. 제거 : p 를 제외한 p 의 모든 배수 지우기
- 4. 반복 : 다음으로 지워지지 않은 숫자를 찾아 2~3번을 반복
- 5. 종료 : $p^2 > n$ 이면 중단하고, 이 때 남은 숫자들은 모두 소수

1.6 Sieve of Eratosthenes

2의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

3의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

1.6 Sieve of Eratosthenes

5의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

7의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

1.6 Sieve of Eratosthenes



```
1  #include <iostream>
2
3  using namespace std;
4
5  const int MAX = 100000;
6  bool is_prime[MAX + 1];
7
8  int main() {
9      // 전부 true로 초기화
10     for(int i = 2; i <= MAX; i++) {
11         is_prime[i] = true;
12     }
13
14     // 소수가 아닌 수 걸러내기
15     for(int i = 2; i * i <= MAX; i++) {
16         if(is_prime[i]) {
17             for(int j = i * i; j <= MAX; j += i) {
18                 is_prime[j] = false;
19             }
20         }
21     }
```

에라토스테네스의 체 | $O(n \log \log n)$

- if(is_prime[i]) : 아직 지워지지 않은 숫자 중 가장 작은 수(p)를 찾기(p는 소수)
- for(int j = i * i; j <= MAX; j += i)
 - j = i * i부터 시작하는 이유? : i * i 이전의 배수들은 이미 false로 변경 됨.
 - ex) 10, 15, 20은 각각 2, 3에서 처리 됨
 - j += i인 이유? : 배수만 제거하기 위해서

1.6 Sieve of Eratosthenes

```
23  int a, b;  
24  
25  cin >> a >> b;  
26  
27  for(int i = a; i <= b; i++) {  
28      if(is_prime[i]) {  
29          cout << i << " ";  
30      }  
31  }  
32  
33  return 0;  
34 }
```

에라토스테네스의 체 | $O(n \log \log n)$

- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

Input: Copy
1 100

Expected Output: Copy
**2 3 5 7 11 13 17 19 23 29 31 37 41
43 47 53 59 61 67 71 73 79 83 89
97**

1.6 Sieve of Eratosthenes



실습(BOJ)

- [1929 소수 구하기](#)

1.6 Sieve of Eratosthenes



실습(정올)

- [1901 소수 구하기](#)
- [1232 에라토스테네스의 체](#)

1. To Be Continued...



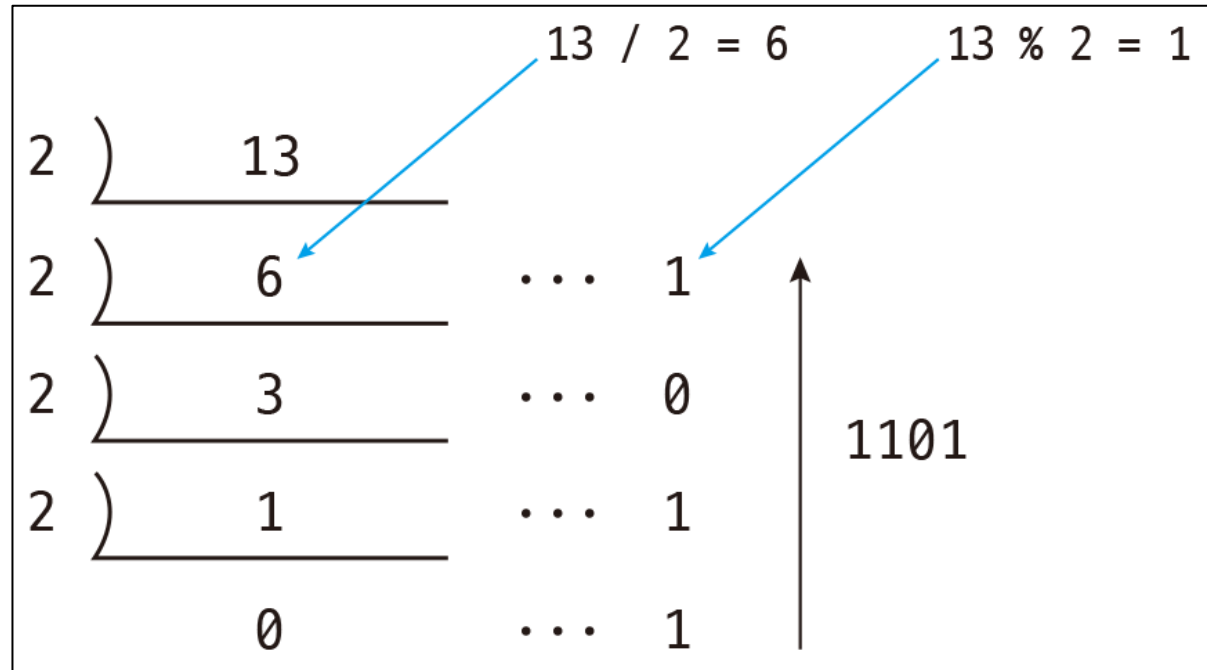
- 오일러 피 함수
- 확장 유클리드 알고리즘
- 중국인의 나머지 정리
- 밀러-라빈 소수 판별법

2. Base Conversion

2.1 Decimal to Base-N

Decimal to Base-N(10진수 → N진수)

- N의 거듭제곱들을 가중치로 갖는 자릿수의 값들을 추출하는 과정
- 10진수 m을 진법 n으로 몫이 0이 될 때까지 나눗셈을 반복하며 나머지를 기록
- 가장 먼저 구한 나머지가 일의 자리가 되므로 기록된 나머지 문자열을 마지막에 역순으로 뒤집어 출력
- 혹은 스택을 이용하여 저장 후 출력



2.1 Decimal to Base-N



- 1534 10진수를 2,8,16진수로

```
1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      int n, m;
9      string s;
10
11     cin >> n >> m;
12
13     while(n > 0) {
14         if(n % m > 9) s.push_back(n % m - 10 + 'A');
15         else s.push_back(n % m + '0');
16
17         n /= m;
18     }
19
20     reverse(s.begin(), s.end());
21
22     cout << s;
23
24     return 0;
25 }
```

2.2 Base-N to Decimal

Base-N to Decimal(N진수 $\rightarrow$ 10진수)

- 각 자릿수의 값에 자릿수 가중치(N의 거듭제곱)를 곱해 모두 더하는 과정
- 우측(일의 자리)부터 좌측으로 순회하며 각 자릿수 값에 자릿수 가중치($1, N, N^2, \dots$)를 곱해 누적
- 구현 시에는 문자열을 역순으로 뒤집은 뒤, 0번 인덱스부터 순회하며 가중치를 매 단계마다 N배씩 곱해 나가며 연산

1	0	1	0	1				
$1 \cdot 2^4$	+	$0 \cdot 2^3$	+	$1 \cdot 2^2$	+	$0 \cdot 2^1$	+	$1 \cdot 2^0$
16	+	0	+	4	+	0	+	1
21								

2.2 Base-N to Decimal



- 2814 이진수

```
1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      string s;
9      long long pow = 1;
10     long long res = 0;
11
12     cin >> s;
13
14     reverse(s.begin(), s.end());
15
16     for(int i = 0; i < s.size(); ++i) {
17         if(s[i] == '1') res += pow;
18
19         pow *= 2;
20     }
21
22     cout << res;
23
24     return 0;
25 }
```

2.3 Horner's Method

Horner's Method(호너의 방법)

- N진수 문자열을 거듭제곱 연산 없이 다항식의 형태를 변형하여 10진수로 변환하는 알고리즘
- 문자열을 뒤집지 않고 거듭제곱 변수없이 왼쪽부터 단 하나의 수식으로 10진수 값을 구할 수 있음
- 문자열을 순회하여 '이전 누적 값에 진법 N을 곱한 뒤 현재 자릿수 값을 더하는 연산'을 반복
- 즉, 매 단계마다 $res = res * N + val$ 연산만 수행하여 $O(N)$ 에 해결

$$\begin{array}{cccc} 8 & 4 & 2 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 1 & 0 & 1 \end{array}$$

$$1x^3 + 1x^2 + 0x^1 + 1$$

기존 방법

$$1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 = 13$$

호너의 방법

$$1x^3 + 1x^2 + 0x^1 + 1 = ((1 * x) + 1) * x + 0 * x + 1$$

$$ax^4 + bx^3 + cx^2 + dx^1 + e = (((ax + b)x + c)x + d)x + e$$

2.3 Horner's Method



```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  string input;
7
8  long long horner(int base) {
9      long long res = 0;
10     for (int i = 0; i < input.size(); i++) {
11         if (input[i] >= 'A' && input[i] <= 'Z') res = res * base + (input[i] - 'A' + 10);
12         else res = res * base + (input[i] - '0');
13     }
14
15     return res;
16 }
17
18 int main() {
19     int base;
20
21     cin >> input >> base;
22
23     cout << horner(base);
24
25     return 0;
26 }
```

2. Base Conversion



실습(정올)

- [8579 프로그래머의 생년월일](#)
- [3106 진법 변환](#)